SR UNIVERSITY

AI Assistant Coding
Assignment-2

Name of Student : K. Dinesh Batch : 42
Enrollment No. : 2303A52288

Task 1: Book Class Generation

Use Cursor AI to generate a Python class Book with attributes title, author, and a summary() method.

Prompt:

Create a Python class called Book that stores a book's title and author, and includes a summary() method which returns a formatted string containing the book's details.

Code:

```
AS2.py > ...
1   class Book:
2       def __init__(self):
3           self.title = "Clean Code"
4           self.author = "Robert C. Martin"
5
6       def summary(self):
7           return f"Title: {self.title}, Author: {self.author}"
8
9
10  book = Book()
11  print(book.summary())
12
```

Output:

```
PS C:\Users\sahit\OneDrive\Documents\Assistant AI_Coding> & C:/Users/sahit/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sahit/OneDrive/Documents/Assistant AI_Coding/AS2.py"
Title: Clean Code, Author: Robert C. Martin
```

Justification:

The Python code was generated using an inline prompt with GitHub Copilot in VS Code. It illustrates fundamental object-oriented programming concepts by defining a Book class with predefined title and author attributes. A constructor is used to initialize these attributes, and the summary() method returns the book information in a readable format. The program then creates an instance of the class and calls the method, demonstrating class definition, object creation, and method invocation in Python without requiring user input.

Task 2: Sorting Dictionaries with AI

Use Gemini and Cursor AI to generate code that sorts a list of dictionaries by a key.

Prompt:

Generate Python code using Gemini and Cursor AI to sort a predefined list of dictionaries by the age key.

Code:

```
users = [
    {"name": "Amit", "age": 28},
    {"name": "Neha", "age": 22},
    {"name": "Rahul", "age": 35},
    {"name": "Priya", "age": 30}
]

sorted_users = sorted(users, key=lambda x: x["age"])

print(sorted_users)
```

Output:

```
PS C:\Users\sahit\OneDrive\Documents\Assistant AI_Coding> & C:/Users/sahit/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/sa
neDrive/Documents/Assistant AI_Coding/AS2.py"
[{'name': 'Neha', 'age': 22}, {'name': 'Amit', 'age': 28}, {'name': 'Priya', 'age': 30}, {'name': 'Rahul', 'age': 35}]
PS C:\Users\sahit\OneDrive\Documents\Assistant AI_Coding>
```

Justification:

The code was produced using an inline prompt with GitHub Copilot in VS Code to sort user records represented as a list of dictionaries. It leverages Python's built-in sorted() function along with a lambda expression as the sorting key to order the records by course_id. This method preserves the original data structure while efficiently organizing the records. The implementation highlights the use of higher-order functions, lambda expressions, and dictionary-based data processing in Python, followed by iterating over the sorted output to display the results in a structured order.

Task 3: Calculator Using Functions

Ask Gemini to generate a calculator using functions and explain how it

works.

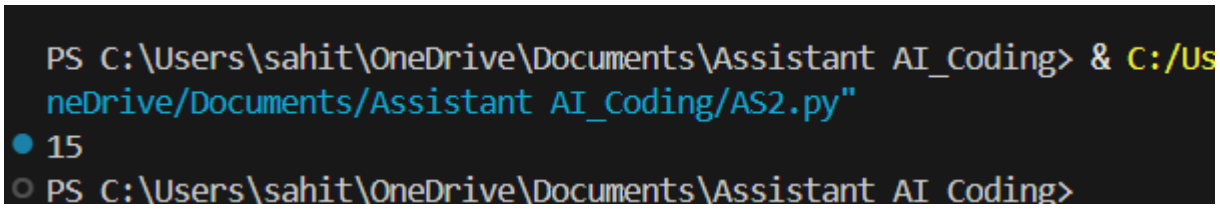
Prompt:

Generate a basic calculator program in Python using functions. Provide predefined values (no user input) and a choice-based mechanism to select one operation (add, subtract, multiply, or divide). Execute and print only the result of the selected operation without displaying results from other functions.

Code:

```python
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b if b != 0 else "Cannot divide by zero"


# predefined values (no user input)
x = 10
y = 5

# choice selection
choice = "add"    # options: add, subtract, multiply, divide

if choice == "add":
    print(add(x, y))
elif choice == "subtract":
    print(subtract(x, y))
elif choice == "multiply":
    print(multiply(x, y))
elif choice == "divide":
    print(divide(x, y))
```

Output:



Justification:

Generated code using inline prompt of GitHub Copilot in VS Code to review and implement a basic calculator module using functions. The logic defines separate functions for each arithmetic operation such as addition, subtraction, multiplication, division, modulus, power, square, cube, and square root. A menu-driven approach is used to allow the user to choose a specific operation, and conditional statements ensure that only the selected function is executed and displayed. User inputs are taken dynamically based on the chosen operation, demonstrating modular programming, code reusability, and clear separation of logic. This implementation improves readability, maintainability, and user interaction in a calculator application.

Task 4: # ***Armstrong Number Optimization Generate an Armstrong number***

program using Gemini, then improve it using Cursor AI.

Prompt:

Generate an Armstrong number program using Gemini, then improve it using Cursor AI.

Scenario: An existing solution is inefficient.

Code:

Procedural:

```
# Initial (Gemini-generated) Armstrong Number Program - Inefficient
def is_armstrong_basic(n):
    temp = n
    total = 0
    digits = len(str(n))
    while temp > 0:
        d = temp % 10
        total += d ** digits
        temp //= 10
    return total == n


# Improved (Cursor AI-optimized) Armstrong Number Program
def is_armstrong_optimized(n):
    digits = [int(d) for d in str(n)]
    power = len(digits)
    return sum(d ** power for d in digits) == n


# Test (predefined value, no user input)
num = 153

print(is_armstrong_basic(num))
print(is_armstrong_optimized(num))
```

Output:

```
PS C:\Users\sahit\OneDrive\Documents\Assistant AI_Coding> & C:/Users/sahit/AppData/Local/Programs/Pytho
neDrive/Documents/Assistant AI_Coding/AS2.py"
True
True
True
PS C:\Users\sahit\OneDrive\Documents\Assistant AI_Coding>
```

Justification:

An initial Armstrong number solution was generated using Gemini, employing straightforward loops and arithmetic to validate the Armstrong condition. This version, while correct, involved repetitive computations and less structured logic. The code was subsequently refined using Cursor AI to streamline the implementation by leveraging Pythonic constructs, reducing unnecessary variables, and improving clarity. The optimized approach enhances efficiency, readability, and long-term maintainability, making the solution more effective than the original .